

■ 2.7.4 Setting Up Contexts for Packages

When you write a package in *Mathematica*, you usually want it to operate in a way that is as independent of its “environment” as possible. You do not want your package to stop working just because the user, or some other package that has been read in, happens to define a particular symbol to have some peculiar value.

The best way to make packages independent is to set up separate contexts for each of them. The standard *Mathematica* convention is to base the names of the contexts on the name used for the file, with the `.m` suffix removed.

<code>System`</code>	built-in <i>Mathematica</i> objects
<code>Package`</code>	objects for export
<code>Package`private`</code>	objects local to the package
<code>Needed₁`</code> , <code>Needed<sub>2</sub>`</code> , ...	other contexts needed for definitions in the package

Types of context used in a typical package.

Two kinds of objects are typically introduced in a package. The first kind are names of functions that are intended to be used outside the package. The second are names of functions and variables that are used only within the package. You can distinguish these two kinds of objects by putting the first ones in a context with a name of the form `Package``, and the second ones in a context with a name of the form `Package`private``. Once you are in the context `Package``, you can refer to the context `Package`private`` using the “relative context” notation ``private``.

There is a standard sequence of context specifications that you can use when you set up a package. The later parts of this section will explain what these context specifications mean. For most purposes, however, you will simply be able to use the template given here.

<code>BeginPackage["Package`"]</code>	make <i>Package`</i> and <i>System`</i> the only active contexts
<code>f::usage = "text", ...</code>	introduce the objects intended for export (and no others)
<code>Begin["`private`"]</code>	begin private context for package
<code>f[args] = value, ...</code>	give the main body of definitions in the package
<code>End[]</code>	end the private context
<code>EndPackage[]</code>	end the package, prepending the context <i>Package`</i> to the global context path

The standard sequence of context control commands in a package.

```
BeginPackage["Collatz`"]

Collatz::usage =
  "Collatz[n] gives a list of the iterates in the 3n+1 problem,
  starting from n. The conjecture is that this sequence always
  terminates."

Begin["`private`"]

Collatz[1] := {1}

Collatz[n_Integer] := Prepend[Collatz[3 n + 1], n] /; OddQ[n] && n > 0
Collatz[n_Integer] := Prepend[Collatz[n/2], n] /; EvenQ[n] && n > 0

End[ ]

EndPackage[ ]
```

The package `Collatz.m`.

Giving `usage` messages at the beginning of a package can be thought of as essentially a trick for introducing in the appropriate context the objects you want to “export” from the package. The point is that when you define these messages, the only symbols you need to mention are exactly the ones you want to export.

When you actually come to give the definitions of the functions in a package, you need to make sure that the local variables and so on are introduced in a “private context” that is not readily accessible from outside the package. By doing this,

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

you can avoid any possibility of confusion between local and global variables, as discussed in Section 2.5.10.

Sometimes you may need to set up a package which uses objects that are defined in other packages. You can tell *Mathematica* to read in the other packages when they are needed.

<pre>Needs["context`"]</pre>	reads in <i>context.m</i> if <i>context`</i> is not in the current list of active contexts
<pre>BeginPackage["Package`", "Needed1`", ...]</pre>	begins a package, reading in other packages that are needed

Functions for specifying interdependence of packages.